

Финальный отчет: Детектирование вредоносных сценариев PowerShell с помощью глубокого обучения

1. Введение и проблематика

В современных кибератаках активно применяются бесфайловые (fileless) угрозы и вредоносные скрипты PowerShell. Традиционные антивирусные решения (на базе сигнатур) часто оказываются неэффективными из-за использования злоумышленниками техник обфускации кода, шифрования полезной нагрузки (Base64) и динамического выполнения. Целью данного проекта является разработка интеллектуального детектора на базе рекуррентных нейронных сетей, способного выявлять вредоносную активность путем анализа структурного и контекстного поведения скрипта, а не его явных сигнатур.

2. Набор данных и разведочный анализ (EDA)

Для обучения и тестирования модели использовался датасет MPSPD_Dataset_v2, содержащий чистые (Benign) и вредоносные (Malicious, включая обфусцированные) скрипты.

В ходе EDA было проанализировано распределение длин скриптов в токенах:

- Выявлено наличие экстремально длинных выбросов, которые могут привести к исчерпанию оперативной (OOM) и видеопамати при обучении.
- **Решение:** Максимальная длина последовательности (Sequence Length) была динамически ограничена по 95-му перцентилю от всего корпуса. Это позволило сохранить максимум полезной информации, отбросив аномалии.

3. Инженерия признаков и предобработка

Для повышения обобщающей способности модели (предотвращения переобучения на конкретных индикаторах компрометации - IoC), был реализован кастомный пайплайн очистки `clean_data_to_tokens`:

1. **Токенизация сущностей:** * Длинные Base64-строки (потенциальные пейлоады) заменялись на токен `[BASE64_PAYLOAD]`.
 - IP-адреса классифицировались и заменялись на `[INTERNAL_IP]` или `[EXTERNAL_IP]`.
 - Hex-значения и длинные числа маскировались токенами `[HEX_VALUE]` и `[LONG_NUMBER]`.
2. **Очистка синтаксиса:** Удаление спецсимволов `()[]{}; ' / \ = : ^ < > | + " & $` для фокуса модели на ключевых командах (cmdlets).
3. **Векторизация (TextVectorization):** Слои словаря размером до 90 000 токенов был интегрирован непосредственно в архитектуру Keras, что гарантирует идентичность преобразования текста на этапах обучения и промышленного инференса.

4. Архитектура модели

Учитывая последовательную природу исходного кода, где значение команды зависит от контекста до и после неё, была выбрана архитектура Bidirectional LSTM (BiLSTM).

Структура сети:

- **Input Layer:** Строковый тензор (raw text).
- **TextVectorization:** Интегрированный слой векторизации.
- **Embedding Layer:** Плотные векторные представления размерностью 32.
- **BiLSTM (64 units) + Dropout (0.3):** Извлечение сложных двунаправленных контекстных связей с возвратом последовательностей.
- **BiLSTM (32 units) + Dropout (0.3):** Агрегация высокоуровневых признаков.
- **Dense (32 units, ReLU) + Dropout (0.3):** Полносвязный слой с регуляризацией для предотвращения переобучения.
- **Output Layer (2 units, Softmax):** Бинарная классификация с выдачей вероятностей классов.

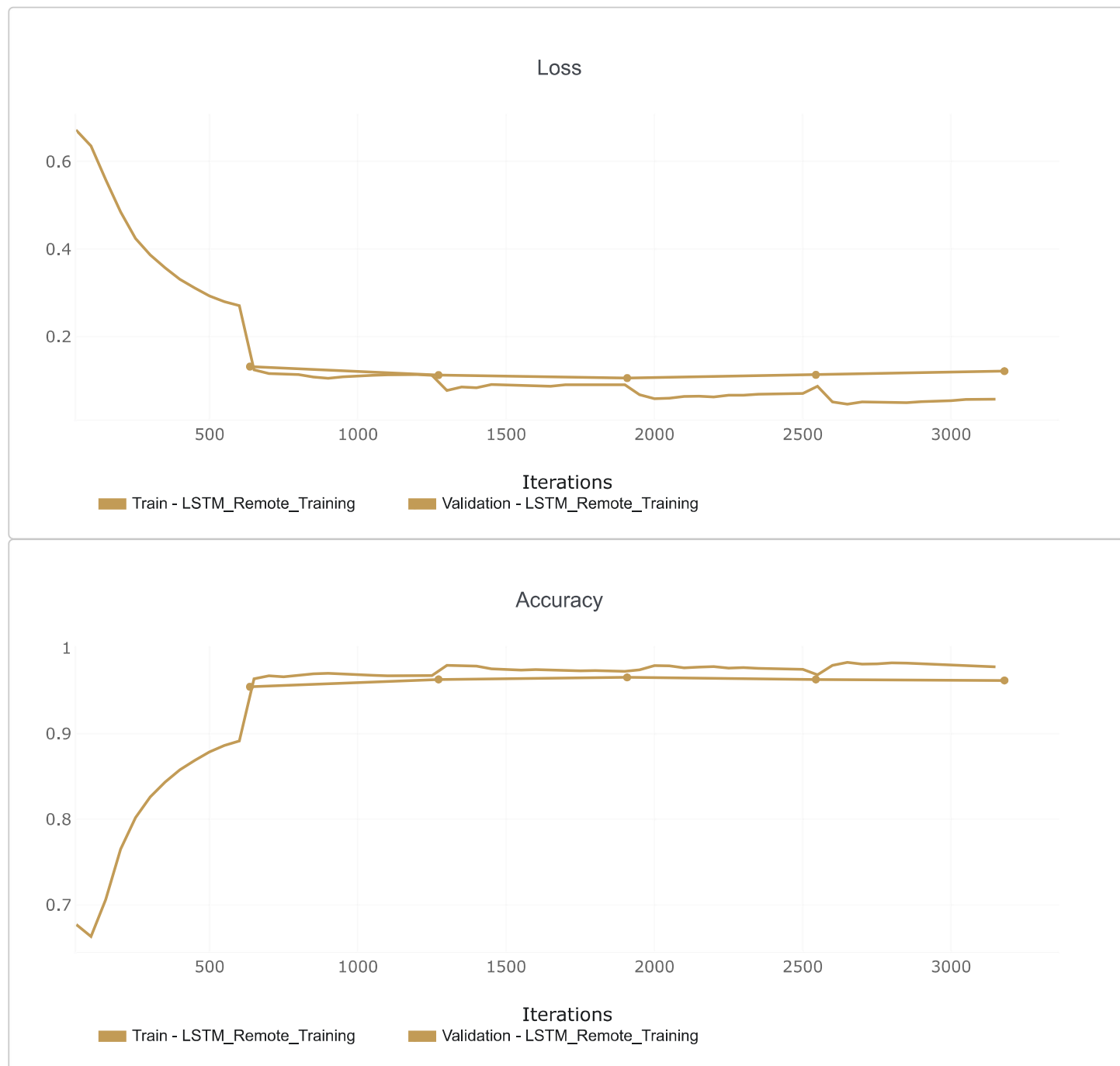
5. Процесс обучения и мониторинг (MLOps)

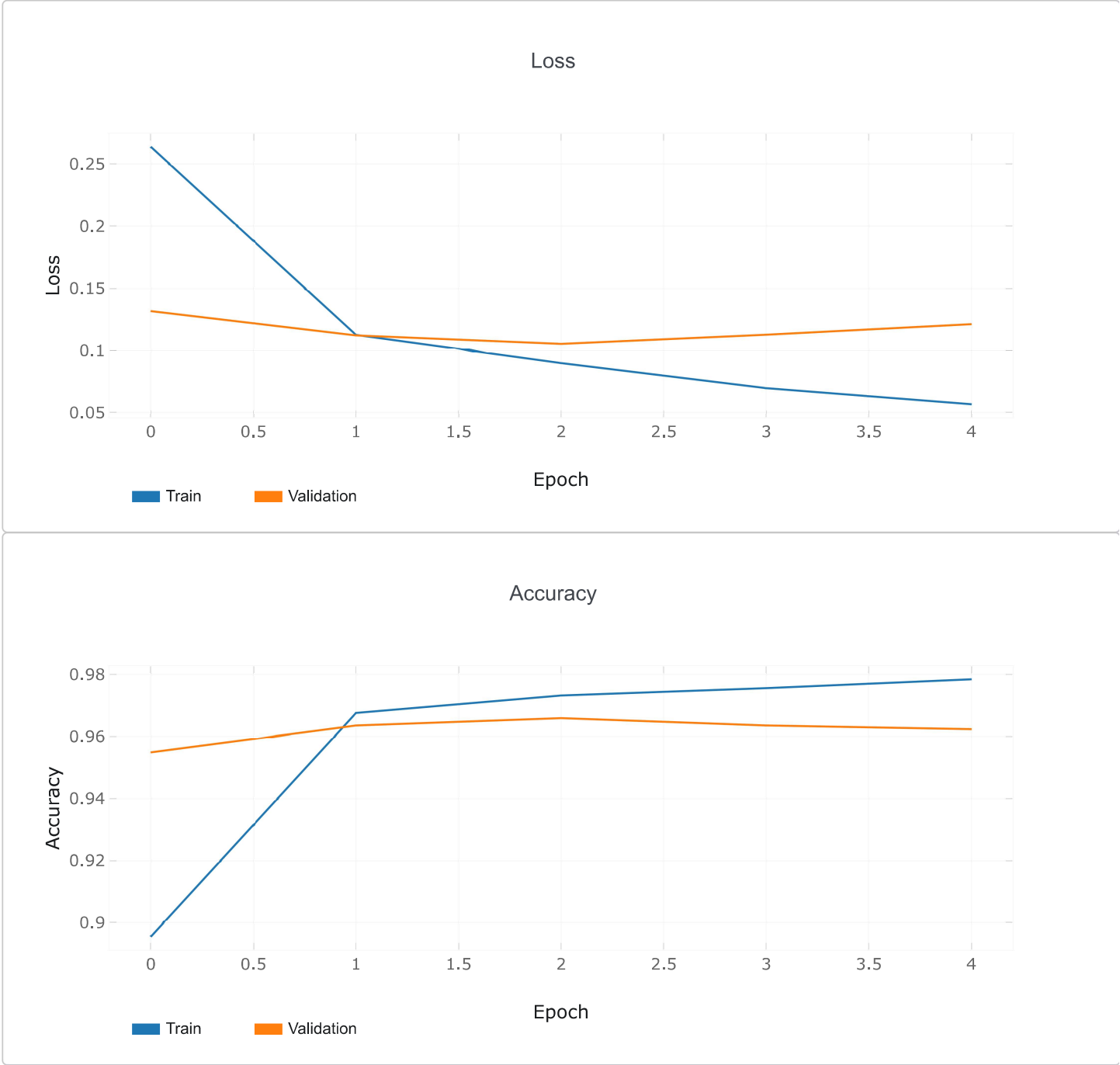
Эксперименты проводились удаленно с использованием ClearML (Task: LSTM_Remote_Training).

- **Оптимизатор:** Adam (Learning Rate = $1e-4$).
- **Функция потерь:** Categorical Crossentropy.
- **Callbacks:** ModelCheckpoint (сохранение лучшей модели по val_accuracy), EarlyStopping (терминация при отсутствии улучшений val_loss в течение 2 эпох), а также кастомный логгер ClearMLBatchLogger для стриминга метрик.

5.1 Динамика обучения

На графиках ниже представлена сходимость функции потерь и рост точности модели по эпохам.





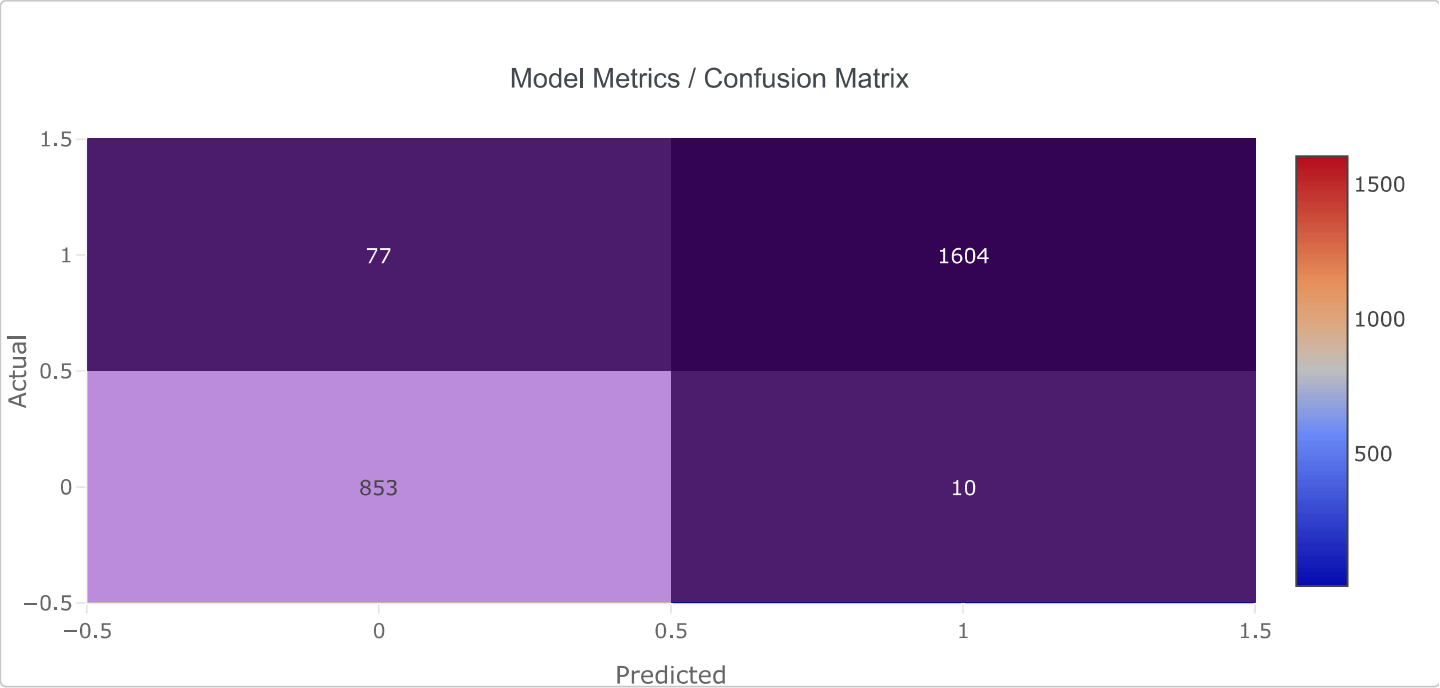
Комментарий к графикам: Модель демонстрирует стабильное обучение без явных признаков переобучения, кривые валидации и обучения сходятся корректно.

6. Результаты и оценка качества

Оценка качества проводилась на отложенной валидационной выборке.

6.1 Матрица ошибок (Confusion Matrix)

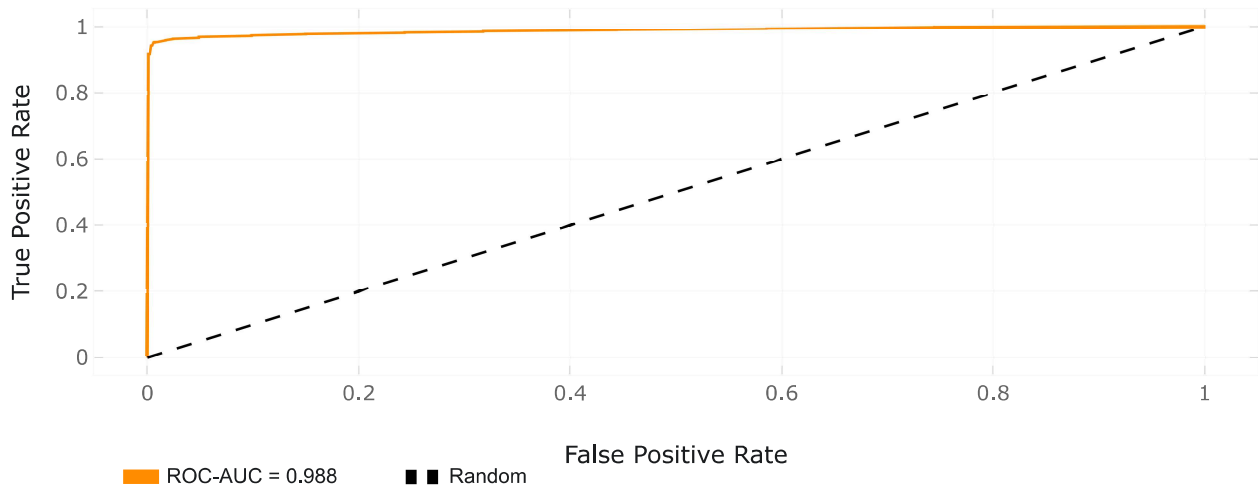
Матрица ошибок позволяет детально оценить количество ложноположительных (False Positives) и ложноотрицательных (False Negatives) срабатываний, что критически важно в ИБ.



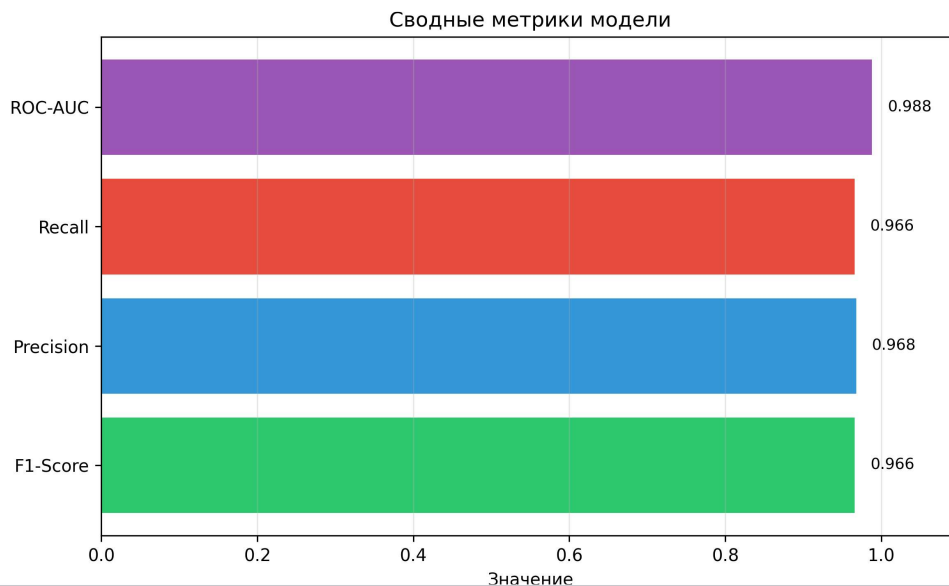
6.2 ROC-Кривая и сводные метрики

Площадь под кривой ошибок (ROC-AUC) демонстрирует высокую разделяющую способность модели.

ROC Curve



Model Metrics/Summary



7. Выводы

В рамках проекта был успешно разработан и обучен нейросетевой детектор вредоносных сценариев PowerShell.

1. Применение специализированной очистки данных и абстрактных мета-токенов позволило модели выучить логику работы вредоносного ПО, сделав ее устойчивой к базовой обфускации.
2. Внедрение слоя TextVectorization внутрь графа модели значительно упростило процесс инференса (`predict.py`).
3. Использование ClearML позволило выстроить прозрачный и легко воспроизводимый MLOps-процесс от подготовки данных до деплоя артефакта модели.